

Архитектура Компьютеров и
операционные системы

Лабораторная работа №12

**Программирование в командном
процессоре ОС UNIX.
Командные файлы**

ОТЧЁТ

МВЕТУВОП ФОНГАНГ ДЖУЛС ФЕРРИ
1132255560

Цель работы

Изучение основ программирования в командной оболочке bash, приобретение навыков написания скриптов для автоматизации задач администрирования и обработки файлов в ОС UNIX/Linux.

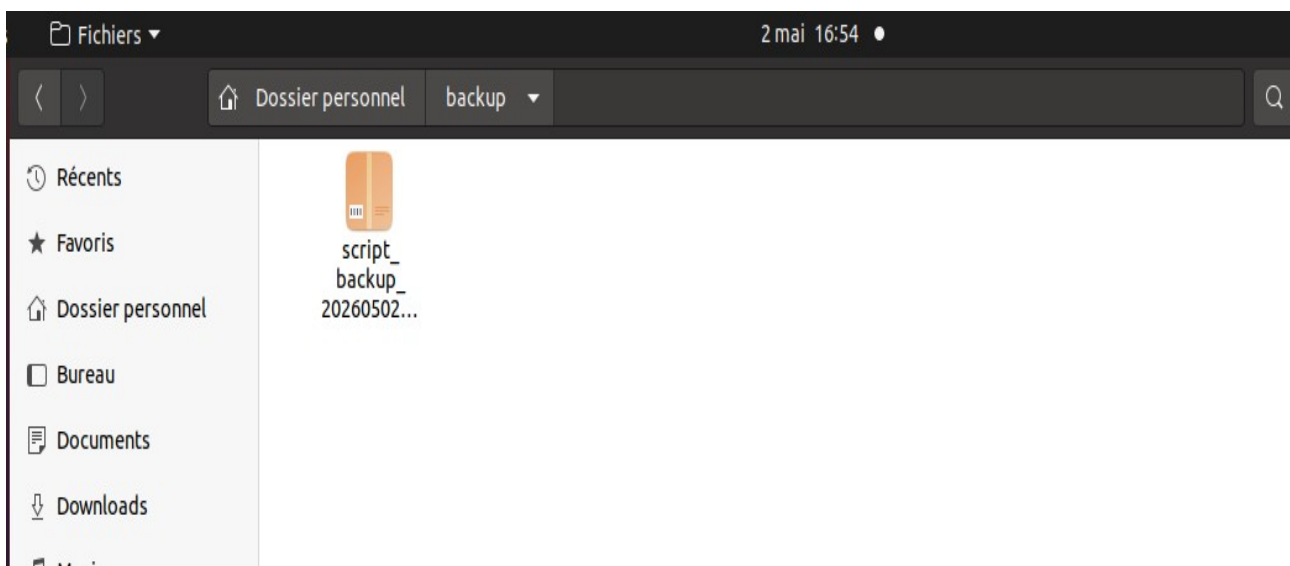
Задание №1. Резервное копирование скрипта

Краткое описание: Создан скрипт, который при запуске создаёт резервную копию своего исходного кода. Копия сохраняется в директории ~/backup с использованием архиватора tar и метки времени в имени файла.



```
1 #!/bin/bash
2
3 # Création du répertoire backup dans le home s'il n'existe pas
4 BACKUP_DIR="$HOME/backup"
5 mkdir -p "$BACKUP_DIR"
6
7 # Nom du script actuel (fichier en cours d'exécution)
8 SCRIPT_NAME="$0"
9
10 # Vérification que le script existe
11 if [ ! -f "$SCRIPT_NAME" ]; then
12     echo "Erreur : Le fichier source n'existe pas."
13     exit 1
14 fi
15
16 # Nom du fichier de sauvegarde avec horodatage
17 DATE=$(date +%Y%m%d_%H%M%S)
18 BACKUP_FILE="$BACKUP_DIR/script_backup_$DATE.tar.gz"
19
20 # Compression avec tar (gzip)
21 tar -czf "$BACKUP_FILE" "$SCRIPT_NAME"
22
23 echo "Sauvegarde effectuée avec succès : $BACKUP_FILE"
24 ls -lh "$BACKUP_FILE"
```

```
fdmvetuvop@jules-N-A:~$ nano script1.sh
fdmvetuvop@jules-N-A:~$ chmod +x script1.sh
fdmvetuvop@jules-N-A:~$ ./script1.sh
Sauvegarde effectuée avec succès : /home/fdmvetuvop/backup/script_backup_20260502_165011.tar.gz
-rw-rw-r-- 1 fdmvetuvop fdmvetuvop 513 mai  2 16:50 /home/fdmvetuvop/backup/script_backup_20260502_165011.tar.gz
fdmvetuvop@jules-N-A:~$
```



Задание №2. Обработка произвольного числа аргументов

Краткое описание: Разработан скрипт, который обрабатывает любое количество аргументов командной строки (более 10). Скрипт выводит общее число аргументов и последовательно печатает каждый аргумент с его порядковым номером.

```
s Éditeur de texte 2 mai 17:01
Ouvrir script2.sh

1 #!/bin/bash
2
3 # Affiche le nombre total d'arguments
4 echo "Nombre total d'arguments : $#"
```

```
5
6 # Affiche chaque argument avec son numéro
7 echo "Liste des arguments :"
```

```
8 count=1
9 for arg in "$@"; do
10     echo "  Argument $count : $arg"
11     ((count++))
12 done
13
```

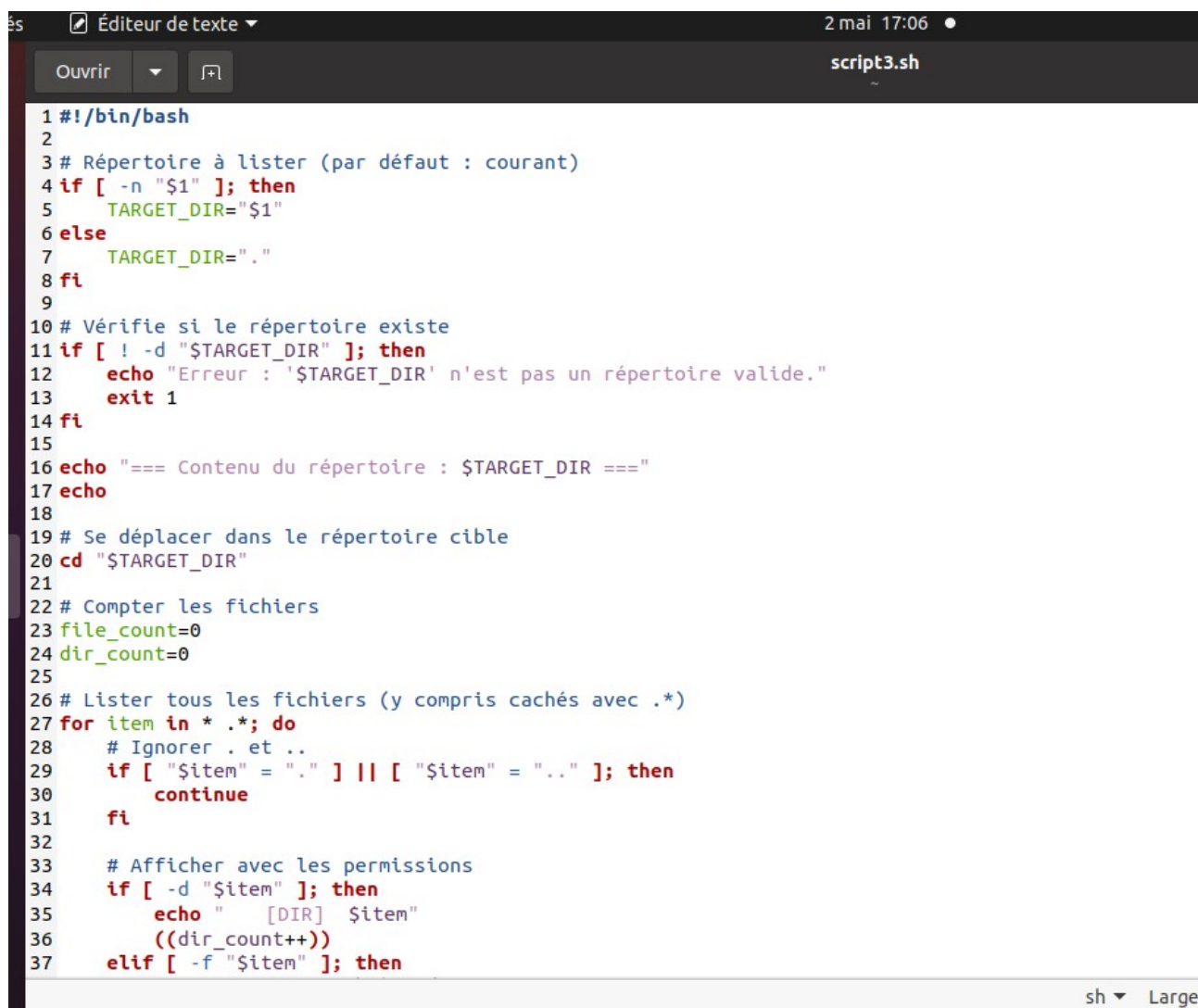
```
14 # Alternative avec une boucle while pour les très grands nombres
15 echo -e "\nMéthode alternative (avec while) :"
```

```
16 i=1
17 while [ $i -le $# ]; do
18     echo "  Argument $i : ${!i}"
19     ((i++))
20 done
```

```
fdmvetuvop@jules-N-A:~$ chmod +x script2.sh
fdmvetuvop@jules-N-A:~$ ./script2.sh a b c d e f g h i j k l m n o
Nombre total d'arguments : 15
Liste des arguments :
  Argument 1 : a
  Argument 2 : b
  Argument 3 : c
  Argument 4 : d
  Argument 5 : e
  Argument 6 : f
  Argument 7 : g
  Argument 8 : h
  Argument 9 : i
  Argument 10 : j
  Argument 11 : k
  Argument 12 : l
  Argument 13 : m
  Argument 14 : n
  Argument 15 : o
```

Задание №3. Аналог команды ls

Краткое описание: Реализован командный файл, который выводит содержимое указанного каталога (по умолчанию — текущего) с информацией о правах доступа. Для определения типа файла используются команды `test -d`, `test -f`, `test -r`, `test -w`, `test -x`. Команды `ls` и `dir` не используются.



```
1 #!/bin/bash
2
3 # Répertoire à lister (par défaut : courant)
4 if [ -n "$1" ]; then
5     TARGET_DIR="$1"
6 else
7     TARGET_DIR="."
8 fi
9
10 # Vérifie si le répertoire existe
11 if [ ! -d "$TARGET_DIR" ]; then
12     echo "Erreur : '$TARGET_DIR' n'est pas un répertoire valide."
13     exit 1
14 fi
15
16 echo "=== Contenu du répertoire : $TARGET_DIR ==="
17 echo
18
19 # Se déplacer dans le répertoire cible
20 cd "$TARGET_DIR"
21
22 # Compter les fichiers
23 file_count=0
24 dir_count=0
25
26 # Lister tous les fichiers (y compris cachés avec .*)
27 for item in * .*; do
28     # Ignorer . et ..
29     if [ "$item" = "." ] || [ "$item" = ".." ]; then
30         continue
31     fi
32
33     # Afficher avec les permissions
34     if [ -d "$item" ]; then
35         echo "[DIR] $item"
36         ((dir_count++))
37     elif [ -f "$item" ]; then
```

```
fdmvetuvop@jules-N-A:~$ chmod +x script3.sh
fdmvetuvop@jules-N-A:~$ ./script3.sh
=== Contenu du répertoire : . ===

rw-  abc1
[DIR]  backup
[DIR]  bin
[DIR]  Bureau
rw-  _config.yml
rw-  conf.txt
[DIR]  Documents
[DIR]  Downloads
rw-  feathers
rw-  file.txt
[DIR]  Images
rw-  lab5.odt
rw-  LICENSE
rw-  may
rw-  mc_4.8.24-2ubuntu1_amd64.deb
rw-  mc-data_4.8.24-2ubuntu1_all.deb
[DIR]  Modèles
[DIR]  mon-site
[DIR]  monthly
[DIR]  Musique
r-x  my_os
[DIR]  play
rw-  presentation.qmd
[DIR]  Public
rw-  quarto-1.7.34-linux-amd64.deb
[DIR]  reports
rwx  script1.sh
rwx  script2.sh
rwx  script3.sh
[DIR]  ski.places
```

Задание №4. Подсчёт файлов по расширению

Краткое описание: Создан скрипт, который принимает два аргумента: расширение файла (например, `.txt`, `.jpg`) и путь к директории. Скрипт подсчитывает количество файлов с указанным расширением в заданной директории.

```
script4.sh
1 #!/bin/bash
2
3 # Vérification du nombre d'arguments
4 if [ $# -ne 2 ]; then
5     echo "Usage : $0 <extension> <chemin_du_repertoire>"
6     echo "Exemple : $0 .txt /home/user/Documents"
7     exit 1
8 fi
9
10 EXTENSION="$1"
11 TARGET_DIR="$2"
12
13 # Si l'extension ne commence pas par ., on l'ajoute
14 if [[ "$EXTENSION" != .* ]]; then
15     EXTENSION=".$EXTENSION"
16 fi
17
18 # Vérifie si le répertoire existe
19 if [ ! -d "$TARGET_DIR" ]; then
20     echo "Erreur : Le répertoire '$TARGET_DIR' n'existe pas."
21     exit 1
22 fi
23
24 # Compter les fichiers avec l'extension donnée
25 COUNT=$(find "$TARGET_DIR" -maxdepth 1 -type f -name "$EXTENSION" 2>/dev/null | wc -l)
26
27 # Alternative avec une boucle (sans find)
28 # COUNT=0
29 # for file in "$TARGET_DIR"/*"$EXTENSION"; do
30 #     if [ -f "$file" ]; then
31 #         ((COUNT++))
32 #     fi
33 # done
34
35 echo "=== Résultat ==="
36 echo "Répertoire : $TARGET_DIR"
37 echo "Extension recherchée : $EXTENSION"
38 echo "Nombre de fichiers trouvés : $COUNT"
```

sh ▾ Largeur des tabulations

```
fdmvetuvop@jules-N-A:~$ chmod +x script4.sh
fdmvetuvop@jules-N-A:~$ ./script4.sh .txt ~
=== Résultat ===
Répertoire : /home/fdmvetuvop
Extension recherchée : .txt
Nombre de fichiers trouvés : 2
fdmvetuvop@jules-N-A:~$
```

Контрольные вопросы

- 1)**Командная оболочка** — это программа, которая позволяет пользователю взаимодействовать с операционной системой, интерпретируя введенные команды. Примеры: sh, bash, csh, ksh, zsh. Они отличаются синтаксисом, набором встроенных функций (история, автодополнение, работа с массивами) и совместимостью со стандартами.
- 2)**POSIX** — это набор стандартов IEEE, описывающих интерфейсы между ОС и прикладными программами, чтобы обеспечить совместимость разных UNIX-систем и переносимость программ на уровне исходного кода.
- 3)**Переменные** в bash определяются присваиванием строки символов без пробелов, например, mark=/usr/bin. **Массивы** создаются командой set -A, за которой следует имя массива и список значений, индексация начинается с нуля.
- 4)**Оператор let** используется для выполнения арифметических вычислений в оболочке. **Оператор read** позволяет считывать значения переменных со стандартного ввода (клавиатуры).
- 5)В bash можно применять арифметические операции: сложение, вычитание, умножение, целочисленное деление, остаток от деления, а также побитовые операции (AND, OR, XOR, сдвиги) и операции сравнения.
- 6)**Операция (())** (двойные круглые скобки) позволяет выполнять арифметические выражения без ключевого слова let, внутри них можно записывать сложные вычисления и присваивания.
- 7)**Стандартные переменные:** PATH (пути поиска команд), HOME (домашний каталог), PS1 и PS2 (приглашения оболочки),

IFS (разделители полей), MAIL (файл для проверки почты), TERM (тип терминала), LOGNAME (имя пользователя).

8)Метасимволы — это специальные символы, имеющие особый смысл для командного процессора, например, звёздочка для любой строки, вопросительный знак для одного символа, знак доллара для подстановки значения переменной.

9)Экранировать метасимволы можно тремя способами: поставить перед метасимволом обратный слеш, заключить строку в одинарные кавычки (полное экранирование) или в двойные кавычки (частичное, кроме символов \$, ' и \).

10)Командные файлы создаются как текстовые файлы с последовательностью команд, начинающиеся со строки `#!/bin/bash`. Запускать их можно после установки права на выполнение командой `chmod +x имя_файла`, затем вводом `./имя_файла`.

11)Функции в `bash` определяются с помощью ключевого слова `function`, за которым следует имя функции и список команд в фигурных скобках, либо просто именем функции и пустыми круглыми скобками.

12)Чтобы определить, является файл каталогом или обычным файлом, используется команда `test` или условный оператор `[ ]` с флагами: `-d` проверяет, является ли файл каталогом, а `-f` — обычным файлом.

13)set отображает все переменные окружения и может устанавливать флаги оболочки. **typeset** объявляет переменные с атрибутами (например, целочисленные). **unset** удаляет переменные или функции (с флагом `-f`).

14)Параметры передаются в командный файл после его имени через пробел. Внутри скрипта доступ к ним осуществляется через `$1`, `$2`, ..., `$9`, а для большего количества используются `${10}`, `${11}` и так далее. `$0` содержит имя самого скрипта.

15)Специальные переменные bash: \$# (число аргументов), \$@ и \$* (все аргументы), \$? (код завершения последней команды), \$\$ (PID текущего процесса), \$! (PID последнего фонового процесса), \${#var} (длина строки переменной), а также OPTARG и OPTIND, используемые с командой getopts.

Выводы

В ходе выполнения лабораторной работы были изучены основные возможности программирования в командной оболочке bash:

1. Написан скрипт для автоматического резервного копирования с использованием архиватора tar
2. Реализована обработка произвольного количества аргументов командной строки
3. Создан аналог команды `ls` с выводом информации о правах доступа
4. Разработан утилита для подсчёта файлов по расширению в указанной директории

В результате были приобретены практические навыки работы с переменными, циклами, условными операторами, функциями и передачей параметров в скриптах bash. Полученные знания могут быть использованы для автоматизации повседневных задач системного администрирования в среде Linux/UNIX.